MATLAB-exercise: Stochastic signal processing (examples of the usage of the correlation function and power spectral density)

0. Purpose and sequence of the exercise

The purpose of the exercise is to study a variety of the classic areas of application for stochastic signal processing in general, and specially correlation functions and power spectral density.

1. Spectral analysis and adaptive systems

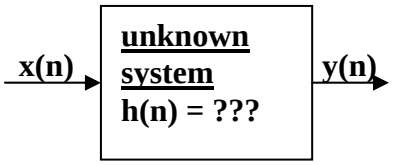
Two of the main applications for stochastic signal processing are:

- estimation of frequency spectra based on a limited amount of signal-samples.
- adaptive systems

These will be studies further in the following exercises.

2. System-identification (non-adaptive)

Here it will be examined, how the cross correlation function can be applied for system identification, i.e. for determination of the frequency characteristics, H(f), or equivalent here to impulse response, h[n], for an unknown system.
The specific application could, for instance, be determination of the frequency characteristics of a transmission wire, which distorts a digital signal or an electrical wire to an acoustic transmission function, distorting an audio-signal.



The set-up is basically the same as in the exercise ”Stochastic signals and linear systems”. Compare the collection of formulas from here, formula (6b), is the method in its most simple version:

$$r_{yx}(k) = \sigma^2 \cdot h(k)$$

a) Add an input-signal consisting of white noise, x(n)
b) Detect the signal on the discharge point, y(n).
c) Determine the cross correlation function between input and output, $r_{yx}(k)$
d) Then the unknown system will be given at $h(k) \cong r_{yx}(k)/ \sigma_x^2$

The method is at real-time DSP-implementation a fine alternative to the classic methods:

- impact the unknown system with a sweep of frequences (slowly)
- impact the unknown system with a short pulse (provides low energy in the output-signal)

A small function is constructed which, for an unknown system H(z) given by numerator coefficients and denominator coefficients in the vectors b and a, partly plots h(n) and H(f) for the ”known system”, partly plots the estimated h(n) and H(f) for the system, determined by means of the above mentioned simple method (so that comparison is possible).

function system id (b,a,N,fs);
% b,a are coefficients for the system, N is the length of the applied white noise input to the system

h=impz(b,a);	the correct impulse response
M=length(h);	
m=0:M-1;	
subplot(4,1,1)	
stem(m,h,'.')	
subplot(4,1,2)	
[H f]=freqz(b,a,1024,fs);	the correct frequency characteristics for the system
plot(f,20*log10(abs(H)))	
subplot(4,1,3)	
x=randn(1,N);	white noise input signal to the system
y=filter(b,a,x);	output-signal from the system
[ryx kyx]=xcorr(y,x,'biased');	cross correlation between input and output
h_est=ryx;	estimated impulse response for the system
stem(kyx,h_est,'.')	
subplot(4,1,4)	
[H_est f]=freqz(h_est,1,1024,fs);	estimated frequency characteristics for the system
plot(f,20*log10(abs(H_est)))	

Unknown system 1: FIR LPF
fs = 8000;
h = fir1(30, 1000/(fs/2));
N = 1024;
systemid(h,1,N,fs)
Use the zoom/scaling-function in plots and compare true/estimated h(n) and H(f). Comments?

Unknown system 2: IIR BPF
[b a] = butter(4, [800/(fs/2) 1200/(fs/2)]);
systemid(b,a,N,fs)
Use the zoom/scaling-function in plots and compare true/estimated h(n) and H(f). Comments?

More complicated/exact method:
If the input-signal is not ”white enough”, this can be taken into account when, instead of resting on (the approached) formula (6b) in the mentioned collection of formulas, using the exact formula (7a): $P_{yx}(f) = H(f)P_{xx}(f)$

When calculating in DSP, the frequency spectras must be determined by DFT (calculated with FFT) and the estimate of the impulse response of the unknow system will then be:

$$h(k) = F^{-1}\{H(f)\} = F^{-1}\left\{\frac{P_{yx}(f)}{P_{xx}(f)}\right\} = F^{-1}\left\{\frac{F\{r_{yx}(k)\}}{F\{r_{xx}(f)\}}\right\} = IFFT\left\{\frac{FFT\{r_{yx}(k)\}}{FFT\{r_{xx}(k)\}}\right\}$$

Note, that for true white noise this method will evolve into the first, simpler method.

The method is tested on the above applied ”unknown” FIR LPF:

```
h = fir1(30, 1000/(fs/2));
y = filter(h,1,x);
[rxx kx] = xcorr(x,'biased');          calculate the correlation function
[ryx kyx] = xcorr(y,x,'biased');
NFFT = 2*N;                            due to the length of the r-vectors
Pxx = fft(rxx,NFFT);                   fft
Pyx = fft(ryx,NFFT);
H_est = Pyx./Pxx;
h_est = real(ifft(H_est,NFFT));        ifft
subplot(2,1,1)
n = 0:NFFT-1;
stem(n,h_est,'.')                      plot estimated h(n)
subplot(2,1,2)
[H f] = freqz(h_est,1,1024,fs);
plot(f,20*log10(abs(H)))               plot estimated H(f)
```

Use the zoom-function and consider the estimate of h(n) and H(f).

3. Pattern recognition / synchronisation / delay-determination

The cross correlation function is capable of retrieving ”hidden” structures in a signal among more random parts and/or noise. Therefore the cross correlation is one of the fundamental methods of pattern recognition, e.g. within the field of image editing.
An one-dimensional application of this is in radar-systems, where a characteristic pulse is transmitted and the reflected, very weak and noise infected, signal is measured. By means of performing cross correlation between sent and received signal, the characteristic pulse form can be recognized in the received signal and be time-delay/distance determined.

Here, ”pattern recognition” is studied in connection with frame synchronization of a long received bit stream, i.e. recognition of a particular bit sequence, a flag which marks where a ”frame” of data starts.

The start flag is defined:

```
Nflag = 20;
flag = ones(1,Nflag);
[rff kf] = xcorr(flag,'none');     the auto correlation function of the flag is determined with
                                   regard to later comparison
stem(kf, rff, '.')
```

A long sequence of random bits (values -1 and +1) is defined and the start flag is placed in a “random spot” in the sequence.

```
Nframe = 1024;
databits1 = sign(rand(1,299)-0.5);          299 random bits with values –1 or +1
databits2 = sign(rand(1,Nframe-299-Nflag)-0.5);
x = [databits1  flag  databits2];           total sent bit sequence
n = 1:Nframe;
stem(n,x,'.')
```

Use zoom-function and check the placing of the flag.

At first no noise is anticipated, i.e. received bit sequence = sent bit sequence ***y = x;***

Cross correlation between received frame and the known flag
[ryf kyf] = xcorr(y,flag,'none');
stem(kyf, ryf,'.')
Specification is performed with the maximum value of the auto correlation of the flag in order to prevent false frame synchronization.
ryf = ryf/max(rff);
stem(kyf,ryf,'.')
Use zoom-function in plot.
Is the position of the flag in the received bit stream precisely determined?
If not, then why not?
You can try with other "seeds" for the rand-coincidence generator.
How could the problem regarding the exact placing of the flag be solved in practical protocols?

Try also using less (more realistic) lengths of the flag, e.g. Nflag = 16, 12, 8.
What can go wrong? There is more than one possibility.
How are the problems solved? There are several classic methods.

In practice, the received digital signal will be noise infected. At detection, i.e. estimation of each received bit to –1 or + 1, errors might occur.
sigma = 0.5;
stoej = sigma*randn(1,Nframe);
y = x + stoej; received signal = sent + noise
y = sign(y); optimal rule of detection with equally probable –1 and +1
[ryf kyf] = xcorr(y,flag,'none');
ryf = ryf/max(rff);
stem(kyf, ryf, '.')

Try increasing the noise level, e.g. for sigma = 1;
Conclusion?

4. Detection of signal in noise

Here we are to study a simple example of how the auto correlation function can be used for detecting a signal which apparently is completely buried in noise.

As sent signal, x(n), a sinus-signal is used in this example, having an unknown frequency f_x.

$$x(n) = A \cdot \sin(2\pi \cdot f_x \cdot n \cdot T_s)$$

The signal effect is then

$$P_x = \left(A/\sqrt{2}\right)^2 = \frac{A^2}{2}$$

A concrete example of such signal is a FSK-signal (Frequency Shift Keying) which is a digital signal, modulated on an analogue carrier wave. The unknown frequency f_x can then adopt a number of values corresponding to the sent symbols/bits. With 2 possible frequencies, the one then corresponds to a sent "0" and the other to a sent "1". This principle of modulation is applied so-called GMSK radio signals in the GSM/DECT-systems.

The received signal is vitiated with Gauss-distributed white noise

$$e(n) = N(0, \sigma^2)$$

with the noise effect

$$P_e = \sigma^2$$

The signal noise condition for the received signal y(n) = x(n) + e(n) is like this

$$SNR = \frac{P_x}{P_e} = \frac{A^2}{2\sigma^2} \qquad og \qquad SNR_{dB} = 10 \cdot \log(SNR)$$

There are various methods for detection of such a noise infected signal, among these we are to study the application of the autocorrelation function as various (and very poor) SNRs are used.

The signal is defined and plotted

subplot(4,1,1) park figure
fs = 8000;
Ts = 1/fs;
N = 256;
n = 0:N-1;
fx = 100; the unknown frequency that is to be detected
A = 1;
x = A*sin(2*pi*fx*n*Ts);
stem(n,x,'.')

The noise is detected and plotted

subplot(4,1,2)
sigma = 1;
e = sigma*randn(1,N);
stem(n,e,'.')

Received signal is defined and plotted

subplot(4,1,3)
y = x + e;
stem(n, y, '.')
SNR = A^2/(2*sigma^2)
SNRdB=10*log10(SNR) please note the low value

Autocorrelation is performed on the received signal

subplot(4,1,4)
[ryy k] = xcorr(y,N/2,'coeff');
stem(k,ryy,'.')

Use zoom-function.

How can the unknown frequency now be determined and <u>to which value</u> on the basis of the autocorrelation function?

Repeat the above mentioned, now with:

1) <u>sigma = 2 corresponding to SNRdB = - 9 dB</u>
2) <u>sigma = 3 corresponding to SNRdB = - 13 dB</u>

Note the extremely poor SNR.

How easily can f_x be detected on the basis of respectively y(n) directly and on the basis of $r_{yy}(k)$?

Comparison to detection with usual BPF:
The useful feature of the autocorrelation in this connection is that it cancels out by averaging the noise in the received signal out into the entire band width of the noise.
However, if a usual BPF is used for detection of fx, then the noise which exists in the pass band of the filter, will still be present at the discharge point of the filter.
This can be illustrated by the following example (for SNRdB = - 13dB)

```
sigma = 3;
e = sigma*randn(1,N);
y = x + e;
subplot(2,1,1)
stem(n,y,'.')
```

A BPF is defined with a pass band e.g. the possible sent frequencies 100, 200, 300, 400Hz:

```
hBP = fir1(50, [50/(fs/2)  450/(fs/2)]);
yBP = filter(hBP,1,y);
stem(n,yBP,'.')
```

How easy is it here to detect f_x ?
Compare with the same SNR-situation for the autocorrelation method.

5. Calculation of correlation functions with Fast Correlation

Real-time calculation of auto- or cross-correlation functions have the same $O(N^2)$ calculation complexity as the convolution operation.
In the same way as convolution can be calculated in the frequency domain with O(NlogN) FFT-based Fast Convolution (as studied previously), the correlation functions can also be calculated in the frequency domain by means of the so-called Fast Correlation.

For the autocorrelation this is based on the relation

$$r_{xx}(k) = x(k) * x(-k)$$

$$P_{xx}(f) = X(f) \cdot X^*(f)$$

This means that the autocorrelation, when using FFT, can be calculated as

$$r_{xx}(k) = IFFT\left\{FFT(x) \cdot FFT^*(x)\right\}$$

Similarly for the cross correlation

$$r_{yx}(k) = x(k) * y(-k)$$

$$P_{yx}(f) = X(f) \cdot Y^*(f)$$

$$r_{yx}(k) = IFFT\left\{FFT(x) \cdot FFT^*(y)\right\}$$

Fast autocorrelation:
A function is constructed, which plots the signal x(n), the ”correct” autocorrelation calculated in the time domain with xcorr, autocorrelation calculated by means of Fast Correlation with default indexing (i.e. r(0), r(1), ...) and finally the last mentioned with the usual plotform for autocorrelation with r(0) in the middle, negative “lags” to the left and positive “lags” to the right.

```
function fastautokorr (x, NFFT);
subplot(4,1,1)
n=0:NFFT-1;
x=[x zeros(1,NFFT-length(x))];
stem(n,x,'.')                          plot signal
subplot(4,1,2)
[rxx k]=xcorr(x);
stem(k,rxx,'.')                        plot autocorr. calculated in the time domain
subplot(4,1,3)
X=fft(x);
rxxfast=ifft(X.*conj(X));              calculate autocorr. in the frequency domain
rxxfast=real(rxxfast);
k=0:NFFT-1;
stem(k,rxxfast,'.')                    plot autocorr. with default-indexing
subplot(4,1,4)
k=[0:NFFT/2 -NFFT/2+1:-1];             make usual indexing for autocorr
stem(k,rxxfast,'.')
```

Test the function with a signal which consists of a pulse:

```
x = ones(1,20);
NFFT = 64;
fastautokorr(x,NFFT)
```

Comments?

In order to make the Fast Correlation work correctly, the same prerequisites must be fulfilled as for Fast Convolution, cf. circular convolution versus linear convolution, zero-padding,....

```
NFFT = 32;
fastautokorr(x,NFFT)
```

Did it go well?
Why not?
Which phenomenon can be observed?

Fast cross correlation:

A corresponding function is constructed, which plots the signals x(n) and y(n), the ”correct” cross correlation calculated in the time domain with xcorr, and the cross correlation calculated by means of Fast Correlation with default indexing and symmetric indexing respectively.

```
function fastkrydskorr (x,y,NFFT);
subplot(5,1,1)
n=0:NFFT-1;
x=[x zeros(1,NFFT-length(x))];
stem(n,x,'.')
subplot(5,1,2)
n=0:NFFT-1;
y=[y zeros(1,NFFT-length(y))];
stem(n,y,'.')
subplot(5,1,3)
[rxy k]=xcorr(x,y);
stem(k,rxy,'.')
subplot(5,1,4)
X=fft(x);
Y=fft(y);
rxyfast=ifft(X.*conj(Y));
rxyfast=real(rxyfast);
```

```
k=0:NFFT-1;
stem(k,rxyfast,'.')
subplot(5,1,5)
k=[0:NFFT/2 -NFFT/2+1:-1];
stem(k,rxyfast,'.')
```

Test the function with a pulse as x(n) and a time displaced pulse as y(n)

```
x = ones(1,20);
y = [zeros(1,10) ones(1,20)];
NFFT = 64;
fastkrydskorr(x,y,NFFT)
```

Conclusion?

```
NFFT = 32;
fastkrydskorr(x,y,NFFT)
```

Conclusion?

6. Possibilities for further experiments

Experiments (possibly) with a simple intuitive method to spectral estimation by means of the cross correlation function:

Define a signal x(n) with a number of sinusoids, e.g. 4-6 frequencies.
Add possible white noise.
Make another signal y(n) with a single sinusoid without noise.
As the frequency of the single-sinus now is varied in appropriately small steps, calculate for each step the cross correlation between the multitone-signal and the single-sinus and the value $\max(r_{yx}(k))$ and the thereto pertaining frequency shall be saved.
The gathered $\max(r_{yx}(k))$-values as function of frequency is an expression of the frequency spectra of the multitone-signal.
Try, plot, conclude.